

AI ASSISTANT CODING

ASSIGNMENT-6.3

TASK 1:

PROMPT

Create a Python program that defines a Student class with the following requirements:

- Attributes: name, roll_number, and branch
- A constructor (__init__) to initialize these attributes
- A method display_details() that prints the student information in a readable format

After defining the class:

- Create at least one Student object
- Call the display_details() method to show the output

CODE

```
# Define a Student class to represent student information
class Student:
    # Constructor to initialize student attributes
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    # Method to display student details
    def display_details(self):
        print(f"Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Branch: {self.branch}")

# Create a sample Student object
student1 = Student("Alice Johnson", 101, "Computer Science")

# Call display_details() to show the student information
student1.display_details()
```

OUTPUT:

```
Name: Alice Johnson
Roll Number: 101
Branch: Computer Science
PS C:\AI assistant>
```

EXPLANATION:

A class Student is defined to represent student data. constructor initializes name, roll_number, and branch. display_details() prints student information in a readable format. An object student1 is created and the method is called to display details. Demonstrates basic OOP concepts: class, object, constructor, and method.

TASK 2 :

PROMPT:Generate a Python function that takes an integer as input and prints the first 10 multiples of that number using a for loop.

Show sample output for a given input.

Explain the loop logic used in the function.

Then generate the same functionality using a while loop instead of a for loop.

Finally, compare both looping approaches in terms of logic, readability, and control.

CODE:

```
# Using for loop
num = int(input("Enter a number: "))

print("First 10 multiples of", num, "using for loop:")
for i in range(1, 11):
    print(num * i)
```

```
# Using while loop
num = int(input("Enter a number: "))

print("First 10 multiples of", num, "using while loop:")
i = 1
while i <= 10:
    print(num * i)
    i += 1
```

OUTPUT:

```
Enter a number: 5
First 10 multiples of 5 using while loop:
5
10
15
20
25
30
35
40
45
50
```

EXPLANATION

Two functions generate the first 10 multiples of a given number. for loop uses range(1, 11) to repeat exactly 10 times. while loop runs until the condition $i \leq 10$ becomes false. Both loops multiply the number with the loop counter. Shows how the same logic can be implemented using different loops.

TASK 3:

PROMPT:

Generate a Python function that classifies a person's age into groups such as child, teenager, adult, and senior using nested if-elif-else conditional statements. After generating the code, explain the conditions and logic used for classification. Then, generate the same age classification using an alternative conditional structure, such as simplified if-else conditions or a dictionary-based approach. Ensure the logic is clear, correct, and easy to understand, and include brief explanations for both approaches.

CODE:

```
def classify_age_nested(age):
    """Classify age into groups using nested if-elif-else statements"""
    if age < 0:
        return "Invalid age"
    elif age < 13:
        return "Child"
    elif age < 18:
        return "Teenager"
    elif age < 65:
        return "Adult"
    else:
        return "Senior"

# Test the function
print("=== Nested If-Elif-Else Approach ===")
test_ages = [5, 15, 30, 70, -5]
for age in test_ages:
    print(f"Age {age}: {classify_age_nested(age)}")

# Alternative: Dictionary-based approach
def classify_age_dict(age):
    """Classify age using a dictionary-based approach"""
    if age < 0:
        return "Invalid age"

    age_groups = {
        (0, 13): "Child",
        (13, 18): "Teenager",
        (18, 65): "Adult",
        (65, 150): "Senior"
    }
```

OUTPUT:

```

=== Nested If-Elif-Else Approach ===
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 70: Senior
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 70: Senior
Age 15: Teenager
Age 30: Adult
Age 70: Senior
Age 30: Adult
Age 70: Senior
Age 70: Senior
Age -5: Invalid age

=== Dictionary-Based Approach ===
Age -5: Invalid age

=== Dictionary-Based Approach ===

=== Dictionary-Based Approach ===
Age 5: Child
Age 15: Teenager
=== Dictionary-Based Approach ===
Age 5: Child
Age 15: Teenager
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 15: Teenager
Age 30: Adult
Age 30: Adult
Age 70: Senior
Age 70: Senior
Age -5: Invalid age
PS C:\AI assitant>

```

EXPLANATION:

`classify_age_nested()` uses if-elif-else to group age logically. Conditions are checked from smallest to largest age range. `HANDLE` invalid age (negative values). `classify_age_alternative()` uses ranges stored in a dictionary. Demonstrates conditional logic and an alternative clean approach.

TASK-4:

PROMPT:

Generate a Python function `sum_to_n(n)` that calculates the sum of the first `n` natural numbers using a for loop. Explain how the generated code works. Show the output for sample inputs like `n = 5` and `n = 10`. Suggest an alternative implementation using: a while loop, and a mathematical formula. Compare all approaches in terms of readability, efficiency, and time complexity.

CODE:

```

# TASK 4: Sum of First n Natural Numbers
# Approach 1: Using for loop
def sum_to_n(n):
    """Calculate sum of first n natural numbers using for loop"""
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
# Test Approach 1
print("=== Approach 1: For Loop ===")
print(f"sum_to_n(5) = {sum_to_n(5)}")
print(f"sum_to_n(10) = {sum_to_n(10)}")
# Approach 2: Using while loop
def sum_to_n_while(n):
    """Calculate sum of first n natural numbers using while loop"""
    total = 0
    i = 1
    while i <= n:
        total += i
        i += 1
    return total
print("\n=== Approach 2: While Loop ===")
print(f"sum_to_n_while(5) = {sum_to_n_while(5)}")
print(f"sum_to_n_while(10) = {sum_to_n_while(10)}")
# Approach 3: Using mathematical formula
def sum_to_n_formula(n):
    """Calculate sum using formula: n * (n + 1) / 2"""
    return n * (n + 1) // 2
print("\n=== Approach 3: Mathematical Formula ===")
print(f"sum_to_n_formula(5) = {sum_to_n_formula(5)}")
print(f"sum_to_n_formula(10) = {sum_to_n_formula(10)}")
# Comparison
print("\n=== Comparison ===")
print("For Loop: O(n) - Clear, readable, intuitive")
print("While Loop: O(n) - Similar to for loop, slightly less clean")
print("Formula: O(1) - Most efficient, fastest, but less obvious")

```

OUTPUT:

```

=== Approach 1: For Loop ===
sum_to_n(5) = 15
sum_to_n(10) = 55

=== Approach 2: While Loop ===
sum_to_n_while(5) = 15
sum_to_n_while(10) = 55

=== Approach 3: Mathematical Formula ===
sum_to_n_formula(5) = 15
sum_to_n_formula(10) = 55

=== Comparison ===
For Loop: O(n) - Clear, readable, intuitive
While Loop: O(n) - Similar to for loop, slightly less clean
Formula: O(1) - Most efficient, fastest, but less obvious
PS C:\AI assitant> 

```

EXPLANATION

for loop adds numbers from 1 to n step by step.while loop does the same using a condition-controlled loop.Formula method uses $n(n+1)/2$ for direct calculation.Formula is the most efficient ($O(1)$), loops are $O(n)$.Shows comparison between iterative and mathematical solutions.

TASK-5

PROMPT:

Generate a Python BankAccount class with the following features:Attributes: account_holder_name and balanceMethods: deposit(amount), withdraw(amount), and check_balance()Ensure withdrawal checks for sufficient balanceAdd meaningful comments explaining each part of the codeDemonstrate the class by creating an object and performing deposit and withdrawal operationsPrint the updated balance after each operation

CODE:

```
class BankAccount:
    """A class to represent a bank account with deposit and withdrawal functions"""

    def __init__(self, account_holder_name, initial_balance=0):
        """
        Constructor to initialize bank account attributes
        Args:
            account_holder_name: Name of the account holder
            initial_balance: Starting balance (default is 0)
        """
        self.account_holder_name = account_holder_name
        self.balance = initial_balance

    def deposit(self, amount):
        """
        Method to deposit money into the account
        Args:
            amount: Amount to deposit
        """
        if amount > 0:
            self.balance += amount
            print(f"✓ Deposited: ${amount:.2f}")
        else:
            print("X Deposit amount must be positive")

    def withdraw(self, amount):
        """
        Method to withdraw money from the account
        Args:
            amount: Amount to withdraw
        """
        if amount > 0 and amount <= self.balance:
            self.balance -= amount
            print(f"✓ Withdrew: ${amount:.2f}")
        elif amount > self.balance:
            print(f"X Insufficient balance. Available: ${self.balance:.2f}")
```

```

        else:
            print("X Withdrawal amount must be positive")

    def check_balance(self):
        """Method to display the current account balance"""
        print(f"Account Balance: ${self.balance:.2f}")

# Demonstrate the BankAccount class
print("=== BankAccount Operations ===")
account = BankAccount("John Doe", 1000)

print(f"\nAccount Holder: {account.account_holder_name}")
account.check_balance()

print("\n--- Deposit Operations ---")
account.deposit(500)
account.check_balance()

account.deposit(250)
account.check_balance()

print("\n--- Withdrawal Operations ---")
account.withdraw(300)
account.check_balance()

account.withdraw(2000)
account.check_balance()

account.withdraw(100)
account.check_balance()

```

OUTPUT:

```

Account Balance: $1350.00
PS C:\AI assitant> & C:/Users/thimi/AppData/Local/Programs/Python/Python311/Python.exe C:\AI assitant\BankAccount.py
=== BankAccount Operations ===

Account Holder: John Doe
Account Balance: $1000.00

--- Deposit Operations ---
✓ Deposited: $500.00
Account Balance: $1500.00
✓ Deposited: $250.00
Account Balance: $1750.00

--- Withdrawal Operations ---
✓ Withdrew: $300.00
Account Balance: $1450.00
X Insufficient balance. Available: $1450.00
Account Balance: $1450.00
✓ Withdrew: $100.00
Account Balance: $1350.00
PS C:\AI assitant> 

```

EXPLANATION:

Represents a real-world bank account using a class. Stores `account_holder_name` and `balance` as attributes. `deposit()` adds money after validating positive amount. `withdraw()` checks for sufficient balance before deduction. Demonstrates encapsulation, validation, and method usage.